Natural Language Processing Lab Report
Task 1: Tokenization and a Guided FAQ Chatbot

Name . MAlik MUsawar ALi

1. Objective

This lab has two parts. In Part A I compare different ways of splitting text into units (whitespace tokens, word-like tokens using regex, and model subwords). In Part B I build a small guided FAQ chatbot that answers questions from text files, handles empty and unrelated questions, and shows possible matches when it is not sure.

2. Part A: Tokenization

I ran six sample texts through two tokenizers. The whitespace tokenizer uses text.split(). The word-like tokenizer uses re.findall(r"\b\w+\b", text), which also works for Urdu because of the UNICODE flag.

Text	Whitespace tokens	Count	Word-like tokens	Count
AI, search!	AI, search!	2	AI search	2
I can't log in.	I can't log in.	4	I can t log in	5
help@example.com	help@example.com	1	help example com	3
یہ ایک مثال ہے	4 Urdu words	4	4 Urdu words	4
(empty string)	none	0	none	0
2.1 version 4944CS	2.1 version 4944CS	3	2 1 version 4944CS	4

What I observed

- "AI, search!" has 2 units in both methods, but whitespace keeps the punctuation (AI, and search!) and the regex removes it.
- "I can't log in." has 4 whitespace units but 5 word-like units, because the regex splits can't into can and t at the apostrophe.
- The email address is 1 whitespace unit but 3 regex units (help, example, com). Which one is better depends on the task: for a search engine the parts are useful, for extracting emails the single unit is better.
- The Urdu sentence is split correctly by both methods since Urdu words are separated by spaces. The version number 2.1 becomes 2 and 1 in the regex, which loses its meaning.
- Empty input gives no units in both methods, and the assert in the code confirms this.

Model subwords (optional comparison)

The code also loads the all-MiniLM-L6-v2 model tokenizer and prints its subword pieces for the first two samples. The subword list depends on the tokenizer version, so I did not guess it. In the environment where I prepared this report the sentence-transformers library was not installed, so the script printed a skipped message instead of crashing. Observed subwords on my machine:

AI, search! -> _____

I can't log in. -> _____

(Run tokenization_demo.py after pip install sentence-transformers and paste the lists here.)

3. Part B: Guided FAQ Chatbot

Design

The FAQ data is stored in text files inside a faq folder. Each file contains blocks of Q: (question) and A: (answer). I expanded the dataset to 15 questions in three files:

- it_help.txt - 7 questions (password, username, WiFi, locked account, software, help desk, two factor)
- library.txt - 4 questions (opening hours, borrowing limit, renewal, fines)
- admissions.txt - 4 questions (documents, last date, fee installment, changing program)

The default backend is lexical (word matching) and works without internet. The steps are:

- Lowercase the text, split into words with the regex, and remove common stopwords like the, how, do, I.
- Turn every FAQ question into a TF-IDF vector (rare words get more weight than common words).
- Turn the user question into a vector the same way and compute cosine similarity with every FAQ question.
- If the best score is 0.50 or more, reply with the answer and the source file name. If it is between 0.15 and 0.50, reply with "Possible matches:" and list the questions. If it is below 0.15, reply "No matching FAQ found". An empty input gives "Please enter a question."

```
def reply(engine, query):
    if not query or not query.strip():
        return "Please enter a question."
    results = engine.search(query)
    if not results or results[0][0] < POSSIBLE_THRESHOLD:
        return "No matching FAQ found. Please contact the help desk."
    score, best = results[0]
    if score >= ANSWER_THRESHOLD:
        return f"{best['answer']}  [source: {best['source']}]"
```

```
    ...  # otherwise list possible matches
```

Verification

I ran task1_chatbot_check.py. It checks four things and all of them passed: (1) a password reset question returns an answer from it_help.txt, (2) empty input returns the please-enter message, (3) volcano geology returns No matching FAQ, and (4) password recovery returns Possible matches.

```
FAQ behaviour checks passed
```

My own transcript

User question	Bot behaviour
How do I reset my password?	Answer from it_help.txt (exact match, score 1.0)
wifi connect karna hai, how do I connect to WiFi	WiFi answer from it_help.txt (mixed language still works because of the English words)
How many books can I borrow?	Borrowing limit answer from library.txt
documents for admission	Documents answer from admissions.txt
What time does the library open?	Possible matches: borrow question and opening hours question (not confident)
password recovery	Possible matches: How do I reset my password? (score 0.45)
(empty)	Please enter a question.
volcano geology	No matching FAQ found
???	No matching FAQ found (no words left after tokenizing)
ہیلو (Urdu hello)	No matching FAQ found (FAQ is English only)

4. Edge Cases and Failure Analysis

- Empty or punctuation-only input: handled without crashing. ??? gives no tokens, so the bot says no match.
- Different word forms: "What time does the library open?" did not get a confident answer, because open and opening are different words for the lexical backend. I did not use stemming. The word time also matched the borrowing question (at one time), which is noise.
- Synonyms: "password recovery" only gets Possible matches since recovery is not in any FAQ question. A semantic model would probably connect recovery with reset, but the assertions in the lab only cover the lexical backend and a semantic backend can still return candidates for unrelated questions.

- Language: Urdu questions get no match because the FAQ is written in English. Adding Urdu FAQ entries would fix this because the tokenizer already handles Urdu text.
- Thresholds: 0.50 and 0.15 are values I chose by testing. They may not be right for a bigger dataset.

5. Connection to Lecture Learning Outcome

Part A shows that the choice of a token (whitespace, regex word, or subword) changes the counts and the information we keep, so tokenization is not a small detail. Part B shows how those tokens become a working retrieval system: tokens, then vectors, then similarity, then a decision rule with a default response when the system is not sure.

6. Reflection and Responsible Practice

What I learned: my first version marked password recovery as a confident match, because the unknown word recovery was ignored in the vector. I fixed it by giving unknown query words a weight, so the score dropped to 0.45. This taught me to test odd inputs and not only the easy ones.

Responsible practice: a chatbot should not pretend to know. My bot says no match or shows possible matches when it is not sure, and it always shows the source file so the user can check the answer. The FAQ answers must be kept correct and up to date, and the bot should not collect personal data such as passwords from users. If I used AI help or the reference code, I should understand it, test it, extend the dataset myself, and say honestly what help I used.

7. Files Submitted

- tokenization_demo.py - Part A, tokenizer comparison
- guided_chatbot.py - chatbot (build_bot, reply)
- task1_chatbot_check.py - the four assertion checks
- transcript.py and transcript.txt - my test conversation
- faq/it_help.txt, faq/library.txt, faq/admissions.txt - FAQ dataset